



Name: CHANDLER MATERE SIMIYU

Admission Number: BSCCS/2024/58571

Unit Title: ADVANCED WEB DESIGN & DEVELOPMENT

Unit Code: BIT3208

Unit Name: Advanced Web Design and Development

Lecturer: MR. NYORO MICHAEL

Year III, Semester II

Project Title: NoteKeeper - Full Stack Web Application

Technologies Used: [Node.js](#) + Express + MySQL

GitHub Link: <https://github.com/Simiyuu/notes-app>

Week 1: Local Environment Setup.....	4
Fig 1: XAMPP Control Panel showing the local development environment installed on the machine.....	4
Fig 2: XAMPP Control Panel showing Apache and MySQL running on their default ports.....	5
Fig 3: Browser showing the XAMPP localhost welcome page.....	5
Fig 4: Hello World Test.....	6
Fig 5: Database Connection Test.....	6
Week 2: Wireframes and GUI Design.....	7
Fig 1: Figma wireframe detailing the planned layout and user interface for the NoteKeeper login page.....	7
Fig 2: Browser showing the Login page with username and password fields used to authenticate registered users.....	8
Fig 3: Mobile View Mockup.....	9
Fig 4: Color Theme and Navigation Structure.....	10
Week 3: JavaScript and PHP Basics.....	11
Fig1: Javascript Form Validation.....	11
Fig 2: Password Strength Checker.....	12
Fig 3: Shows the Express.js route syntax used in server.js to handle HTTP requests.....	13
Fig 4 shows the database connection configuration and initDB function in server.js.....	14
Fig 5: Dynamic User Input Handling.....	14
Week 4 Task Schedule.....	15
Task 1 - Understanding Server-Side Programming.....	15
Fig 1: Simple Processing Page.....	15
Fig 2: Dynamic Welcome Page.....	15
Fig3: Form Submission System.....	16
Task 2 - HTML Forms and PHP Integration.....	17
Fig 1: Registration Form.....	17
Fig 2: Login Form.....	17
Task 3 - Simple Authentication.....	18
Fig 1: Basic Login Page.....	18
Fig 2: Username/passwords validation.....	19
Fig 3: Session-based welcome page.....	20
Task 4 - Backend Folder Organization.....	20
Fig 1: Project Structure.....	20
Task 5 - Introduction to Modern Backend Alternatives.....	21
Fig : PHP Modern Alternative (Node.js).....	21
Week 5 - Advanced Web Design and Development.....	22
Task 1: Database Creation.....	22
Fig 1: notes-app database.....	22
Task 2: Creating Tables.....	22
Fig 1: notes-app tables.....	22

Task 3: CRUD Operations.....	23
Fig 1: Add Records (INSERT).....	23
Fig 2: View Records (SELECT).....	23
Fig 3: Delete Records (DELETE).....	23
Task 4: Connecting PHP to Database.....	24
Fig 1: Database Connection.....	24
Task 5: Fetching Database Records.....	24
Fig 1: Database Records.....	24
Creativity: Note Counter.....	25
Fig 1: shows the dynamic note count badge in the sidebar header displaying the total number of notes.....	25

Week 1: Local Environment Setup

Fig 1: XAMPP Control Panel showing the local development environment installed on the machine

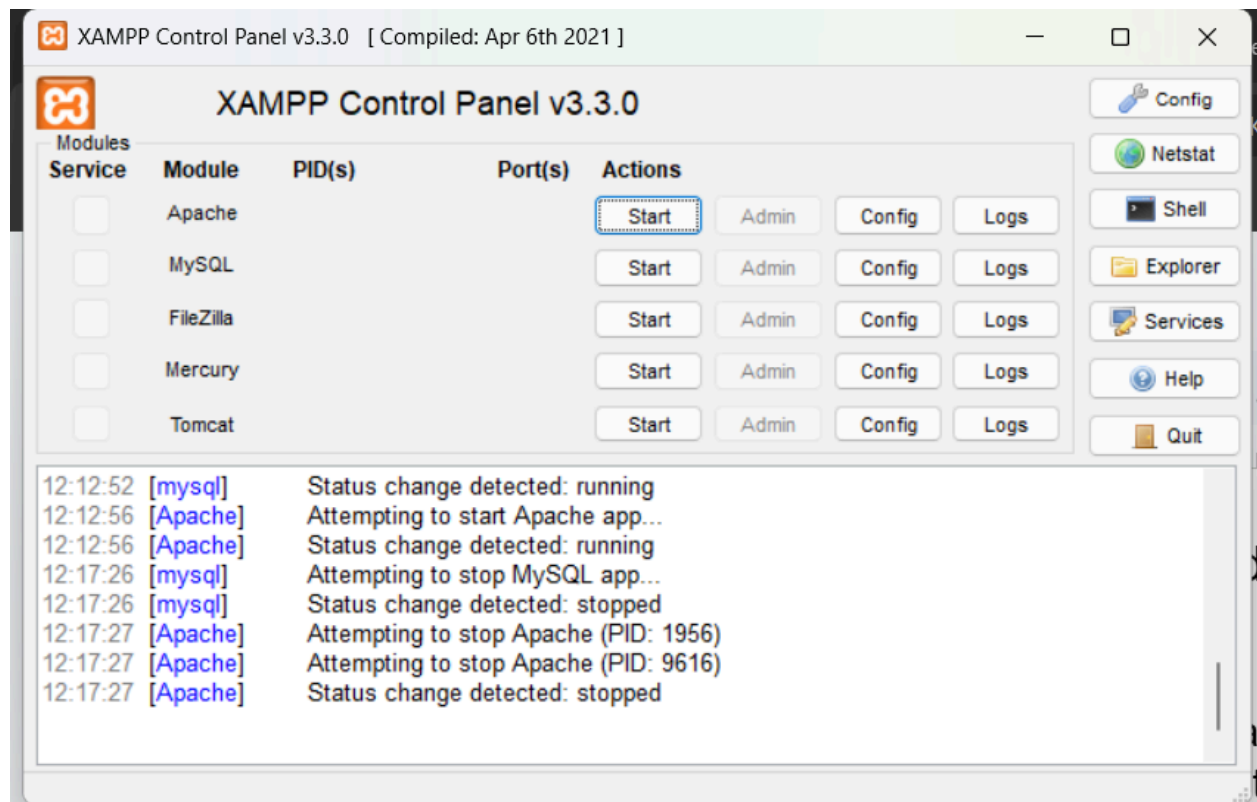


Fig 2: XAMPP Control Panel showing Apache and MySQL running on their default ports

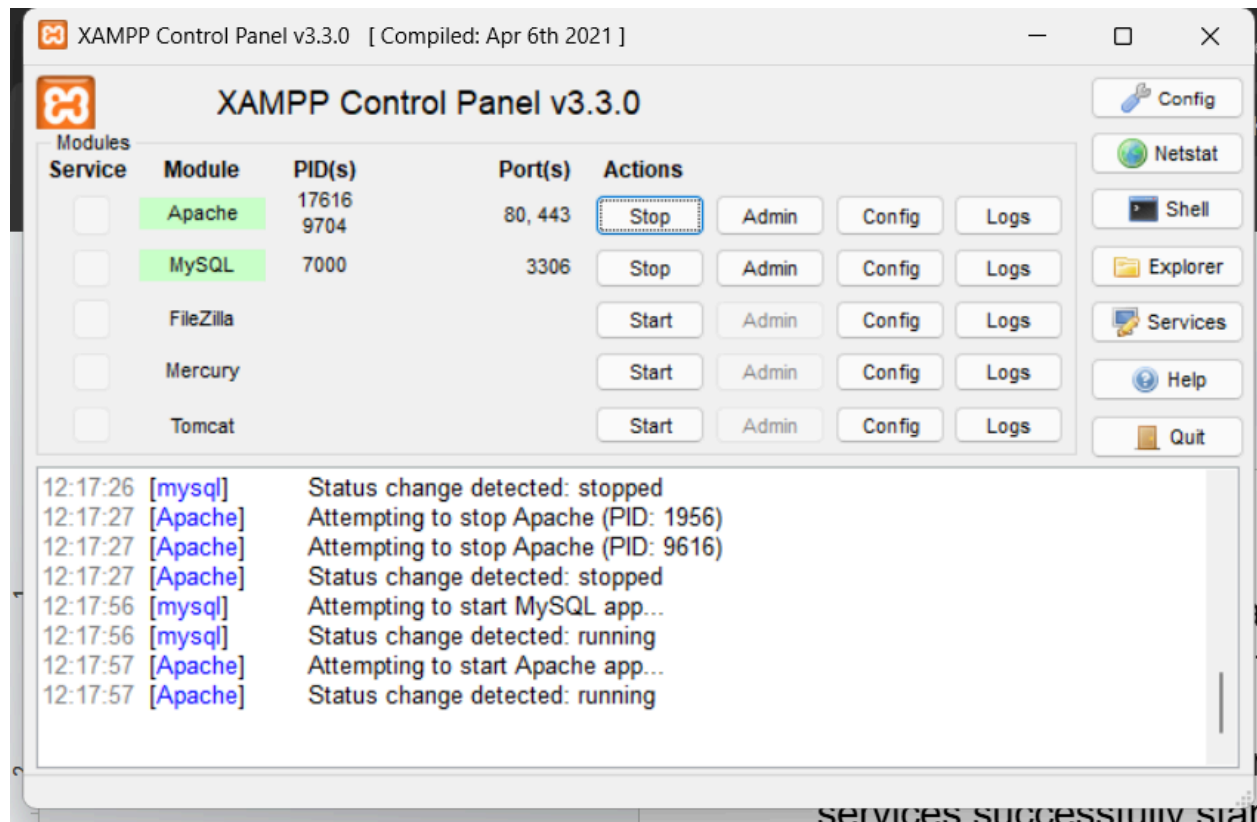


Fig 3: Browser showing the XAMPP localhost welcome page

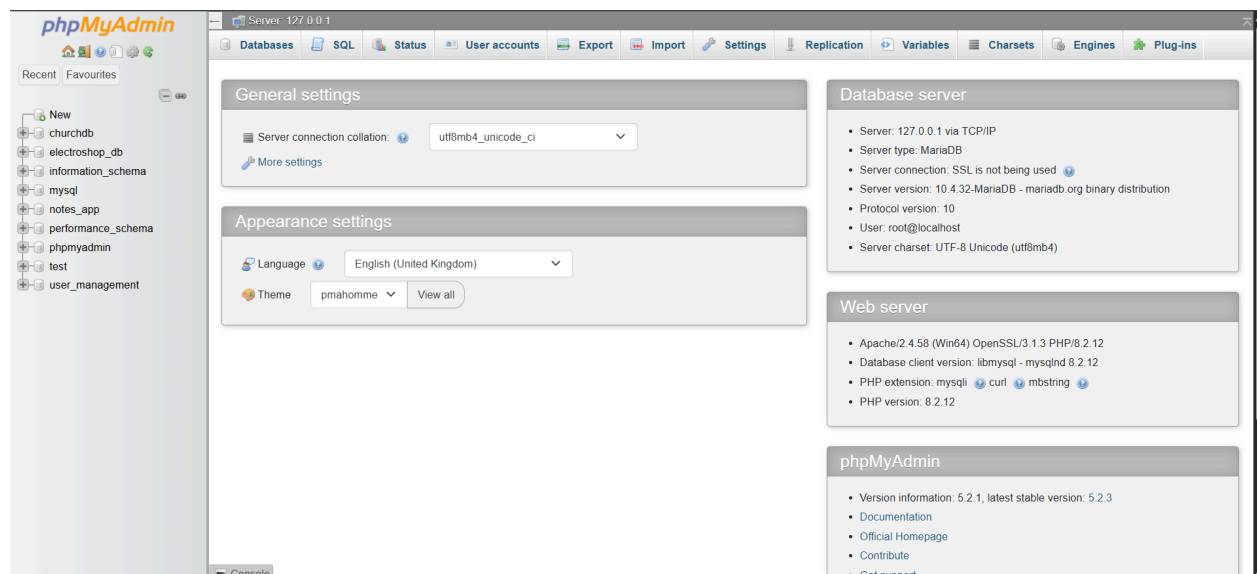


Fig 4: Hello World Test

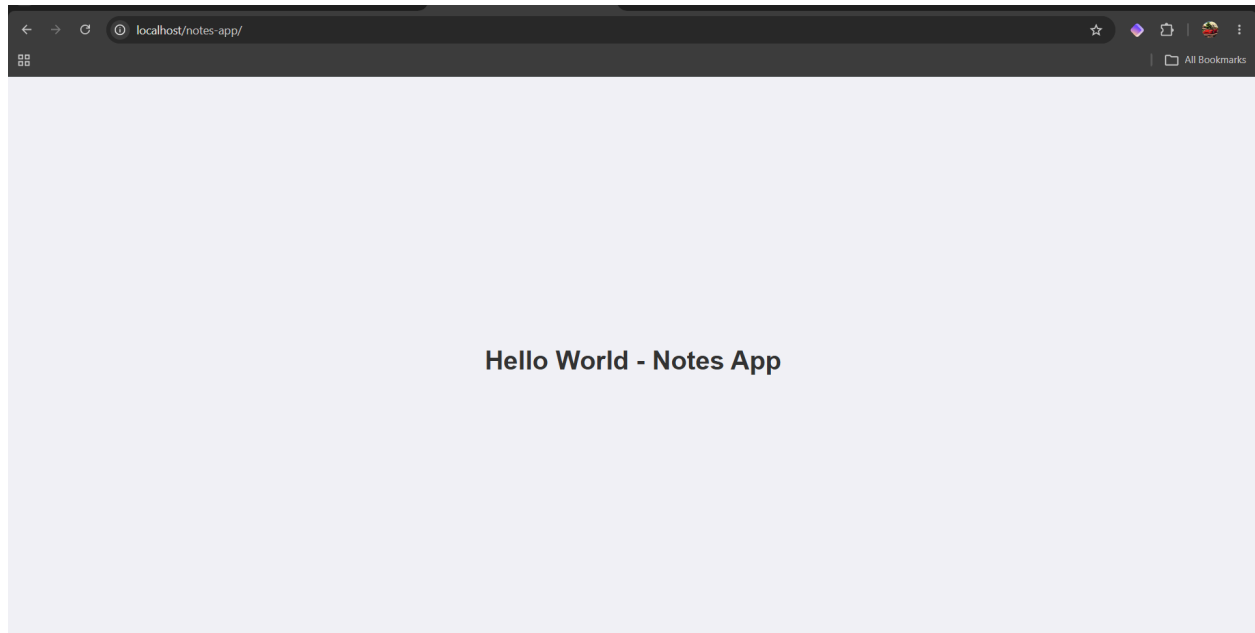
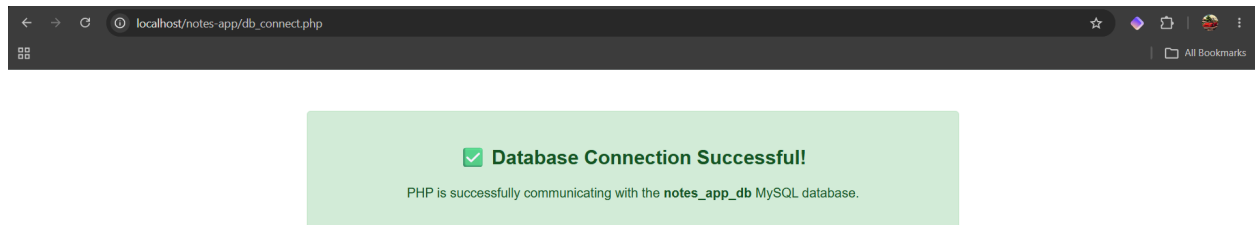


Fig 5: Database Connection Test



Week 2: Wireframes and GUI Design

Fig 1: Figma wireframe detailing the planned layout and user interface for the NoteKeeper login page.

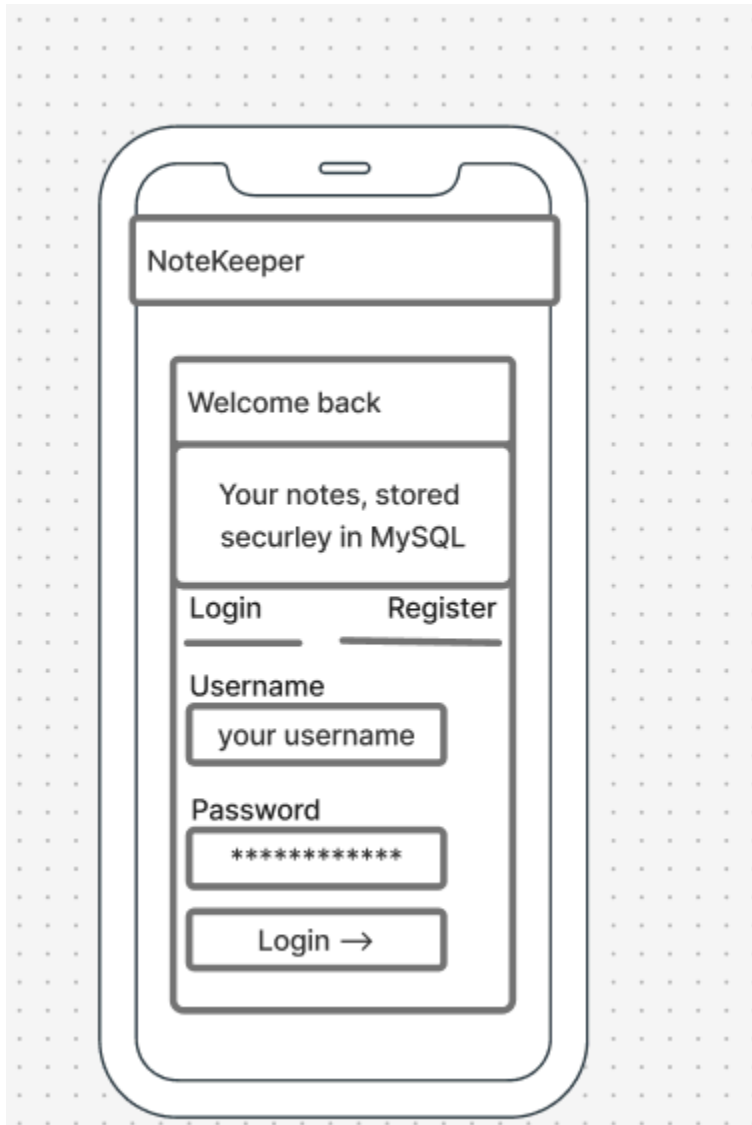


Fig 2: Browser showing the Login page with username and password fields used to authenticate registered users

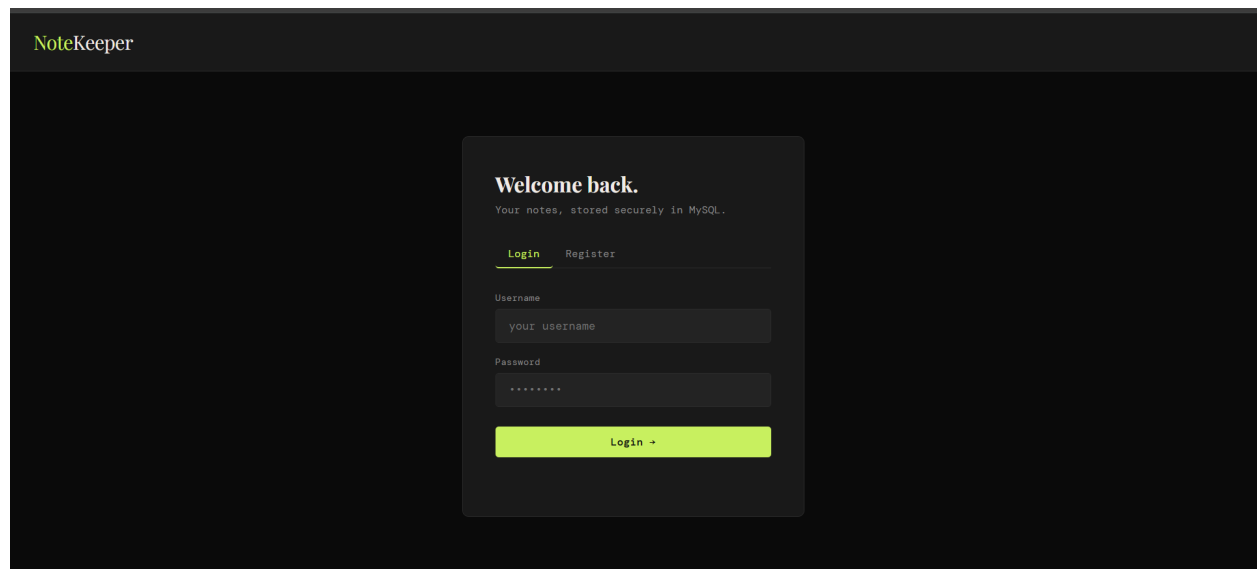
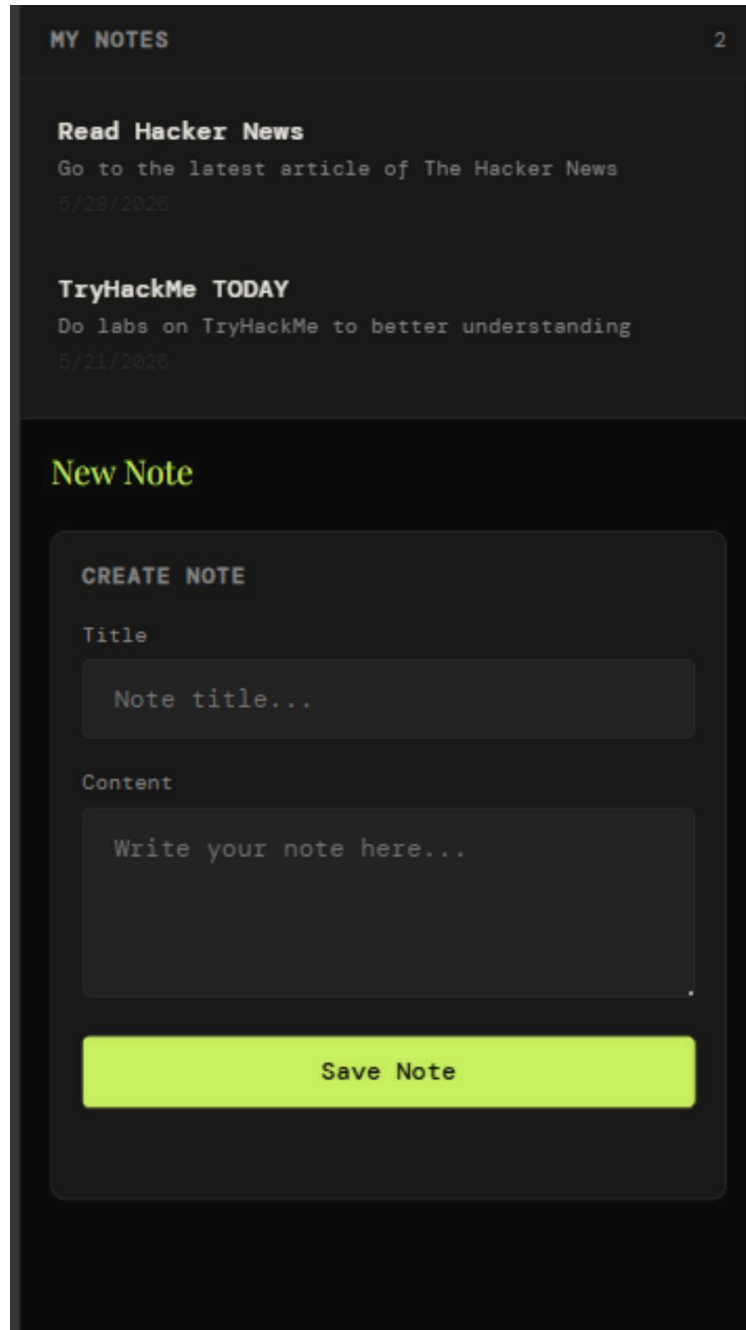


Fig 3: Mobile View Mockup



The mockup shows a mobile application interface with a dark theme. At the top, a header bar contains the text "MY NOTES" on the left and the number "2" on the right. Below the header, there are two note entries. The first entry is titled "Read Hacker News" and contains the text "Go to the latest article of The Hacker News" with a date "5/28/2026" below it. The second entry is titled "TryHackMe TODAY" and contains the text "Do labs on TryHackMe to better understanding" with a date "5/21/2026" below it. Below these entries, there is a section titled "New Note" in a light green font. This section contains a form for creating a new note. The form has a title "CREATE NOTE" and two input fields: "Title" and "Content". The "Title" field has a placeholder text "Note title..." and the "Content" field has a placeholder text "Write your note here...". Below the input fields, there is a green button labeled "Save Note".

MY NOTES 2

Read Hacker News
Go to the latest article of The Hacker News
5/28/2026

TryHackMe TODAY
Do labs on TryHackMe to better understanding
5/21/2026

New Note

CREATE NOTE

Title

Note title...

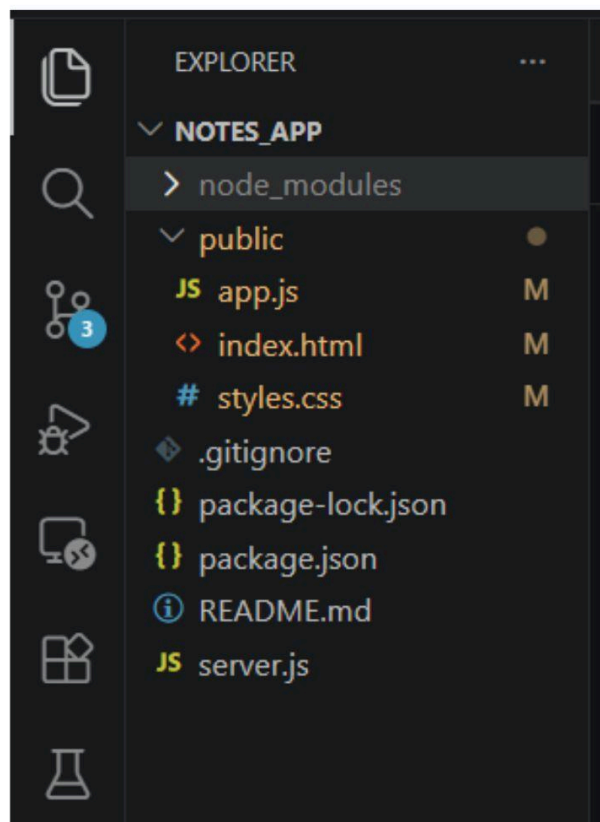
Content

Write your note here...

Save Note

Fig 4: Color Theme and Navigation Structure

	Background
	Surface
	Accent
	Text
	Muted
	Error



Week 3: JavaScript and PHP Basics

Fig1: Javascript Form Validation

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

Email

Password

Create Account →

Please fill in all fields

Fig 2: Password Strength Checker

Welcome back.

Your notes, stored securely in MySQL.

[Login](#) [Register](#)

Username

linet

Email

linetkamathi@gmail.com

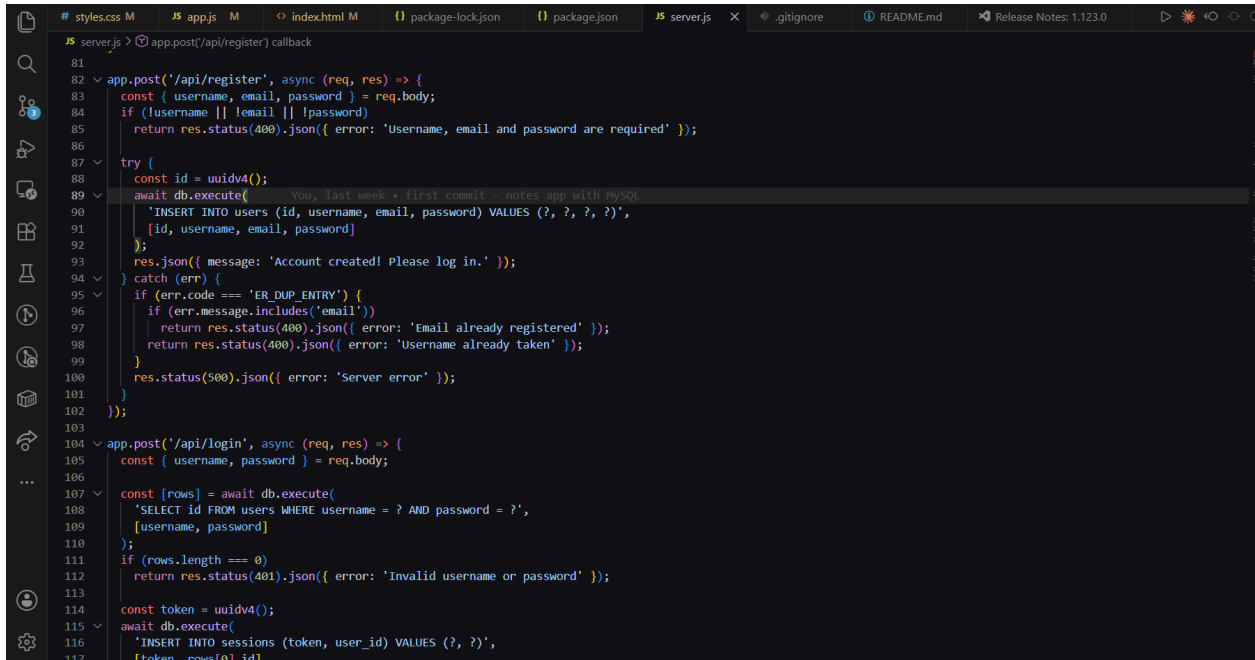
Password

.....

Medium

Create Account →

Fig 3: Shows the Express.js route syntax used in server.js to handle HTTP requests

A screenshot of a code editor with a dark theme. The editor shows a file named 'server.js' with JavaScript code for an Express.js application. The code includes two main routes: a POST route for '/api/register' and a POST route for '/api/login'. The register route uses 'app.post' and 'async' functions, and it includes database operations using 'await db.execute'. The login route also uses 'app.post' and 'async' functions, and it includes database operations. The editor has a sidebar on the left with various icons for file management and development tools. The top of the editor shows several open files: 'styles.css', 'app.js', 'index.html', 'package-lock.json', 'package.json', 'server.js', '.gitignore', 'README.md', and 'Release Notes: 1.123.0'. The 'server.js' file is currently selected and its content is displayed in the main editor area.

```
1 # styles.css M JS app.js M index.html M package-lock.json package.json JS server.js X .gitignore README.md Release Notes: 1.123.0
2 JS server.js > app.post('/api/register') callback
3
4 81
5 82 app.post('/api/register', async (req, res) => {
6 83   const { username, email, password } = req.body;
7 84   if (!username || !email || !password)
8 85     return res.status(400).json({ error: 'Username, email and password are required' });
9 86
10 87   try {
11 88     const id = uuidv4();
12 89     await db.execute(
13 90       'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
14 91       [id, username, email, password]
15 92     );
16 93     res.json({ message: 'Account created! Please log in.' });
17 94   } catch (err) {
18 95     if (err.code === 'ER_DUP_ENTRY') {
19 96       if (err.message.includes('email'))
20 97         return res.status(400).json({ error: 'Email already registered' });
21 98       return res.status(400).json({ error: 'Username already taken' });
22 99     }
23 100     res.status(500).json({ error: 'Server error' });
24 101   }
25 102 });
26 103
27 104 app.post('/api/login', async (req, res) => {
28 105   const { username, password } = req.body;
29 106
30 107   const [rows] = await db.execute(
31 108     'SELECT id FROM users WHERE username = ? AND password = ?',
32 109     [username, password]
33 110   );
34 111   if (rows.length === 0)
35 112     return res.status(401).json({ error: 'Invalid username or password' });
36 113
37 114   const token = uuidv4();
38 115   await db.execute(
39 116     'INSERT INTO sessions (token, user_id) VALUES (?, ?)',
40 117     [token, rows[0].id]
```

Fig 4 shows the database connection configuration and initDB function in server.js.

```
const DB_CONFIG = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: '',
  database: 'notes_app',
  multipleStatements: true
};

let db;

async function initDB() {
  const tempConn = await mysql.createConnection({
    host: DB_CONFIG.host,
    port: DB_CONFIG.port,
    user: DB_CONFIG.user,
    password: DB_CONFIG.password
  });

  await tempConn.execute(`CREATE DATABASE IF NOT EXISTS \`${DB_CONFIG.database}\``);
  await tempConn.end();

  db = await mysql.createPool(DB_CONFIG);
}
```

Fig 5: Dynamic User Input Handling

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds)

SELECT * FROM `notes`

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

	id	user_id	title	content	created_at
Edit Copy Delete 14f49b21-31f0-4204-aafd-8f7d2e02a11a 635e5f65-f7fe-4e86-b56d-9a9086dfcf09 Machine Learning How Kenya has integrated Machine Learning 2026-06-03 12:37:30	Edit Copy Delete 9d568126-c31e-45a2-b458-8d103f3f9c43 635e5f65-f7fe-4e86-b56d-9a9086dfcf09 TryHackMe Do labs on TryHackMe to better understanding 2026-05-21 13:25:24	Edit Copy Delete 9f938a0c-f96-4a76-bdc0-674a8319eb31 635e5f65-f7fe-4e86-b56d-9a9086dfcf09 Read Hacker News Go to the latest article of The Hacker News 2026-05-28 14:33:14			

Week 4 Task Schedule

Task 1 - Understanding Server-Side Programming

Fig 1: Simple Processing Page

```
app.post('/api/register', async (req, res) => {
  const { username, email, password } = req.body;
  if (!username || !email || !password)
    return res.status(400).json({ error: 'Username, email and password are required' });
  // You, last week * first commit - notes app with MySQL
  try {
    const id = uuidv4();
    await db.execute(
      'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
      [id, username, email, password]
    );
    res.json({ message: 'Account created! Please log in.' });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') {
      if (err.message.includes('email'))
        return res.status(400).json({ error: 'Email already registered' });
      return res.status(400).json({ error: 'Username already taken' });
    }
    res.status(500).json({ error: 'Server error' });
  }
});
```

Fig 2: Dynamic Welcome Page

The screenshot shows the NoteKeeper web application. At the top, the title "NoteKeeper" is on the left, and "logged in as chendler" with a "logout" button is on the right. The main content area is divided into two sections. On the left, under the heading "MY NOTES" with a count of "3", there is a list of three notes: "Machine Learning" (dated 07/06/2020), "Read Hacker News" (dated 07/06/2020), and "TryHackMe" (dated 07/06/2020). On the right, under the heading "New Note", there is a "CREATE NOTE" form. The form has two input fields: "Title" (with placeholder text "Note title...") and "Content" (with placeholder text "Write your note here..."). Below the form is a yellow "Save Note" button.

Fig3: Form Submission System

Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

linet

Email

linetkamathi@gmail.com

Password

.....

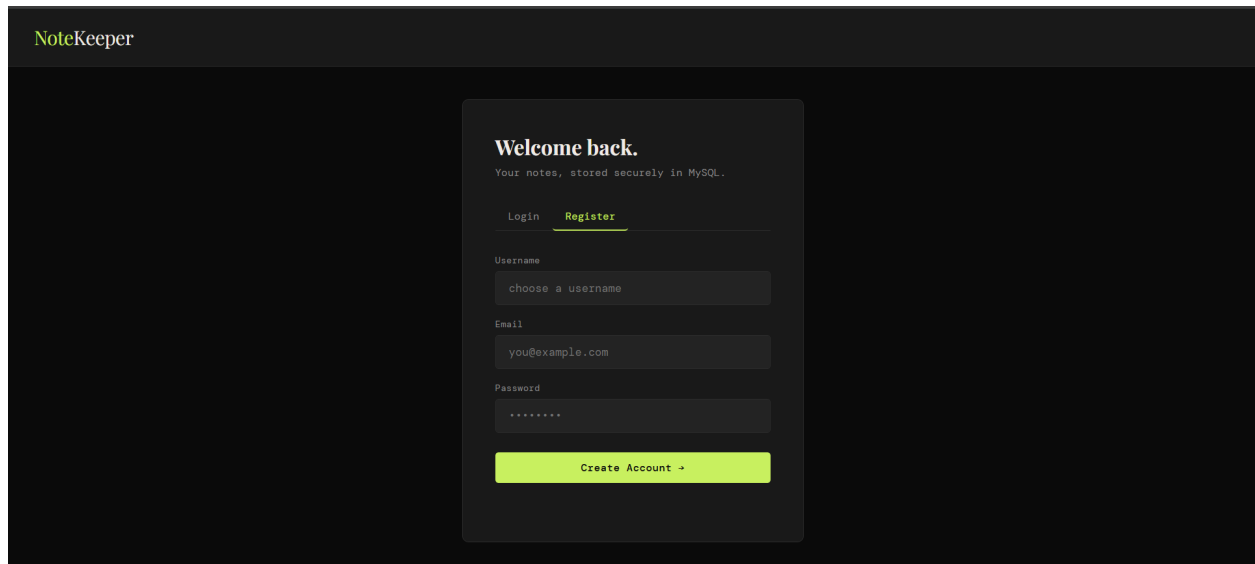
Medium

Create Account →

✓ Account created! Please log in.

Task 2 - HTML Forms and PHP Integration

Fig 1: Registration Form



The screenshot shows the registration form of the NoteKeeper application. The header features the 'NoteKeeper' logo. The main content area has a dark background with a central white card. The card displays a 'Welcome back.' message and a sub-header 'Your notes, stored securely in MySQL.'. Below this, there are two tabs: 'Login' and 'Register', with 'Register' being the active tab. The form fields include 'Username' (placeholder: 'choose a username'), 'Email' (placeholder: 'you@example.com'), and 'Password' (placeholder: '*****'). A green 'Create Account ->' button is positioned at the bottom of the form.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

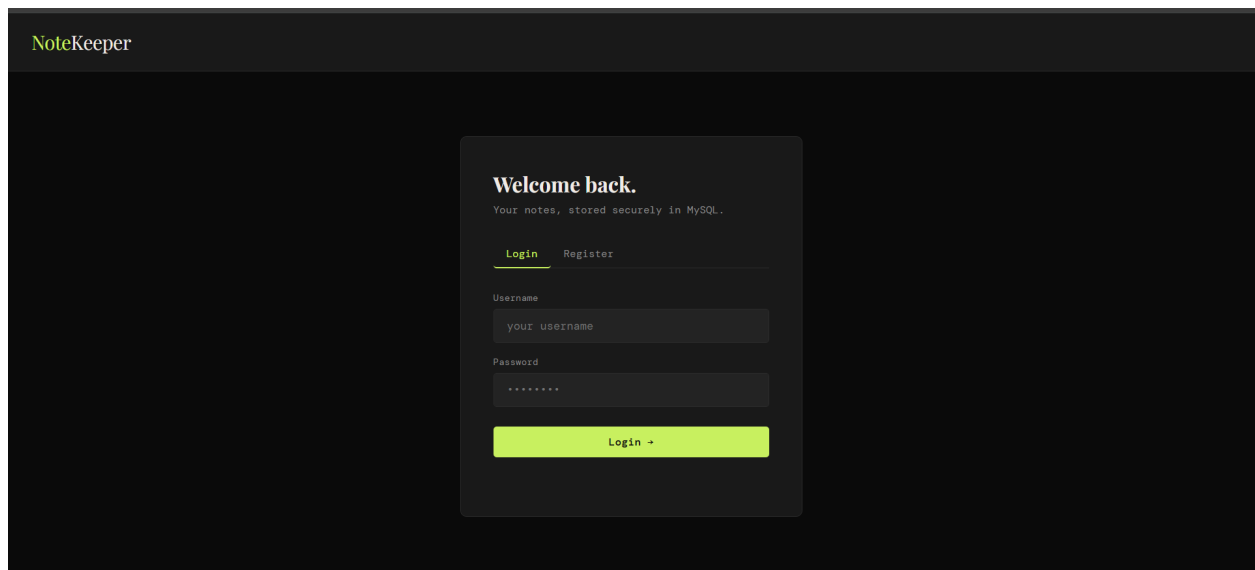
Username
choose a username

Email
you@example.com

Password

Create Account ->

Fig 2: Login Form



The screenshot shows the login form of the NoteKeeper application. The header features the 'NoteKeeper' logo. The main content area has a dark background with a central white card. The card displays a 'Welcome back.' message and a sub-header 'Your notes, stored securely in MySQL.'. Below this, there are two tabs: 'Login' and 'Register', with 'Login' being the active tab. The form fields include 'Username' (placeholder: 'your username') and 'Password' (placeholder: '*****'). A green 'Login ->' button is positioned at the bottom of the form.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

Login Register

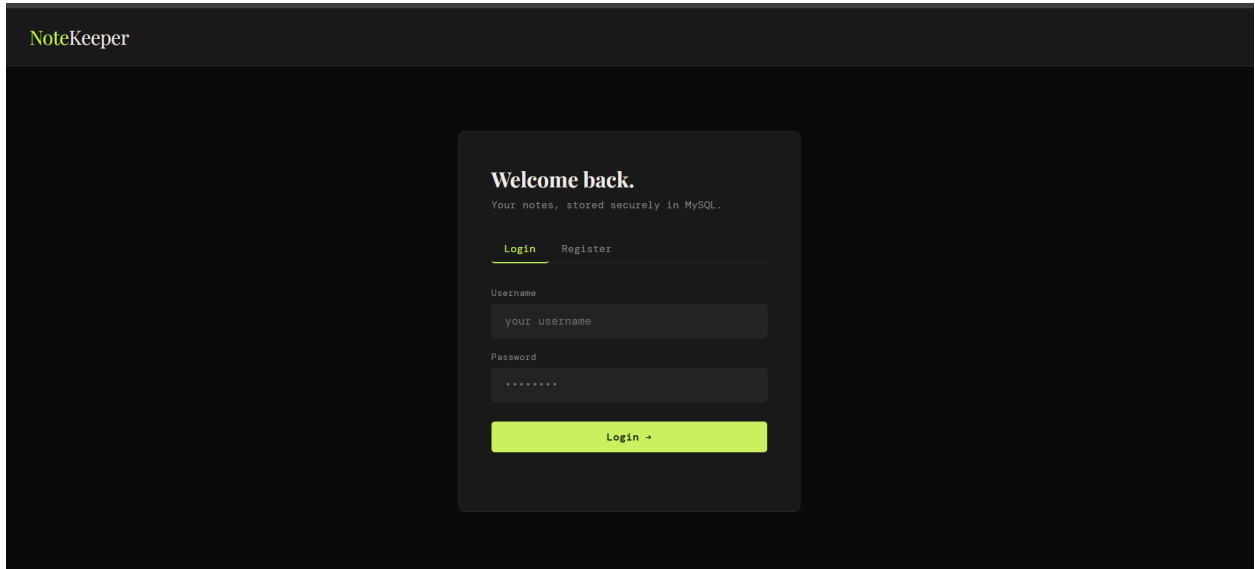
Username
your username

Password

Login ->

Task 3 - Simple Authentication

Fig 1: Basic Login Page



The image shows a web application interface for 'NoteKeeper'. The header is dark gray with the 'NoteKeeper' logo in the top left corner. The main content area is black, featuring a central white login card. The card has a heading 'Welcome back.' followed by the text 'Your notes, stored securely in MySQL.'. Below this, there are two links: 'Login' (underlined) and 'Register'. The 'Login' section includes a 'Username' label, a text input field containing 'your username', a 'Password' label, a password input field with masked characters, and a bright green 'Login →' button.

NoteKeeper

Welcome back.
Your notes, stored securely in MySQL.

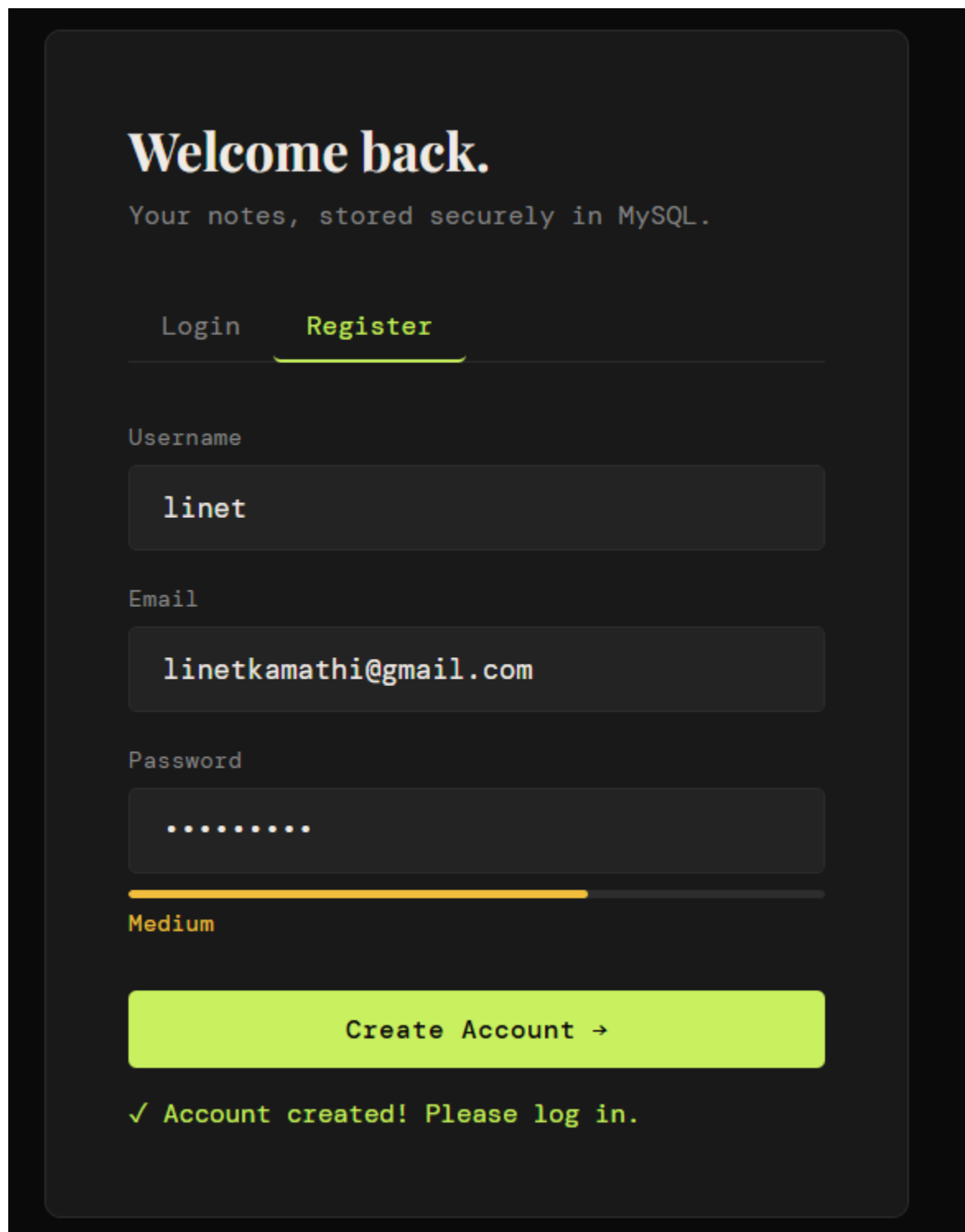
[Login](#) [Register](#)

Username
your username

Password

Login →

Fig 2: Username/passwords validation



Welcome back.

Your notes, stored securely in MySQL.

Login **Register**

Username

linet

Email

linetkamathi@gmail.com

Password

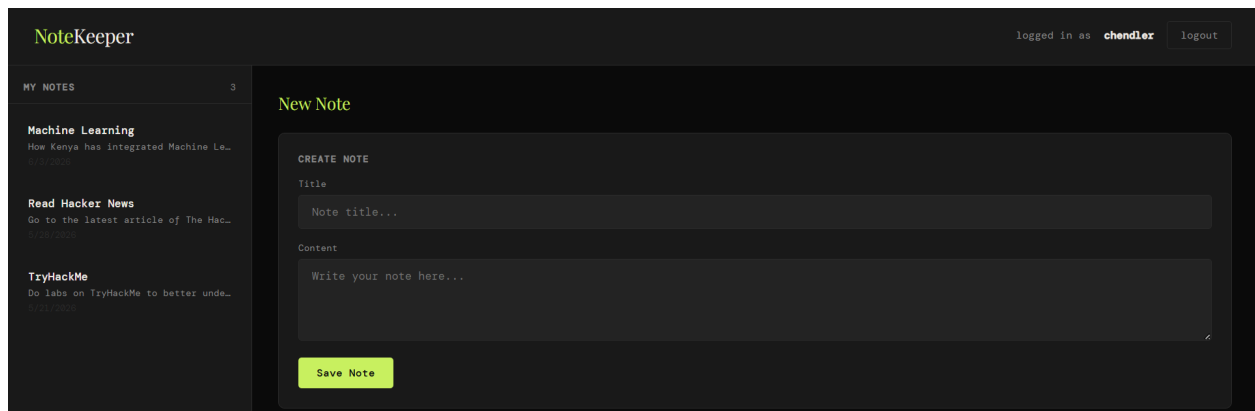
.....

Medium

Create Account →

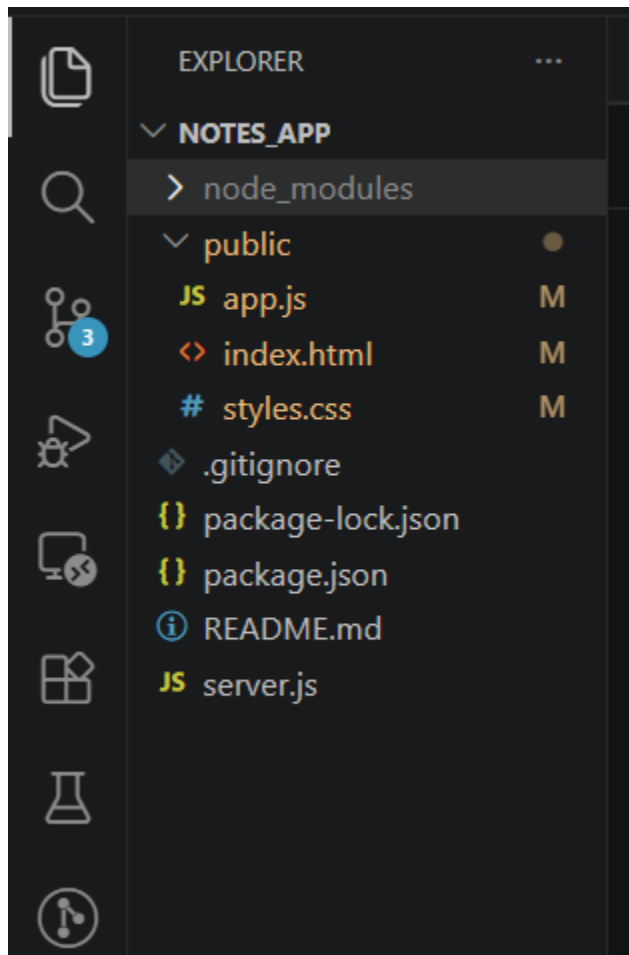
✓ Account created! Please log in.

Fig 3: Session-based welcome page



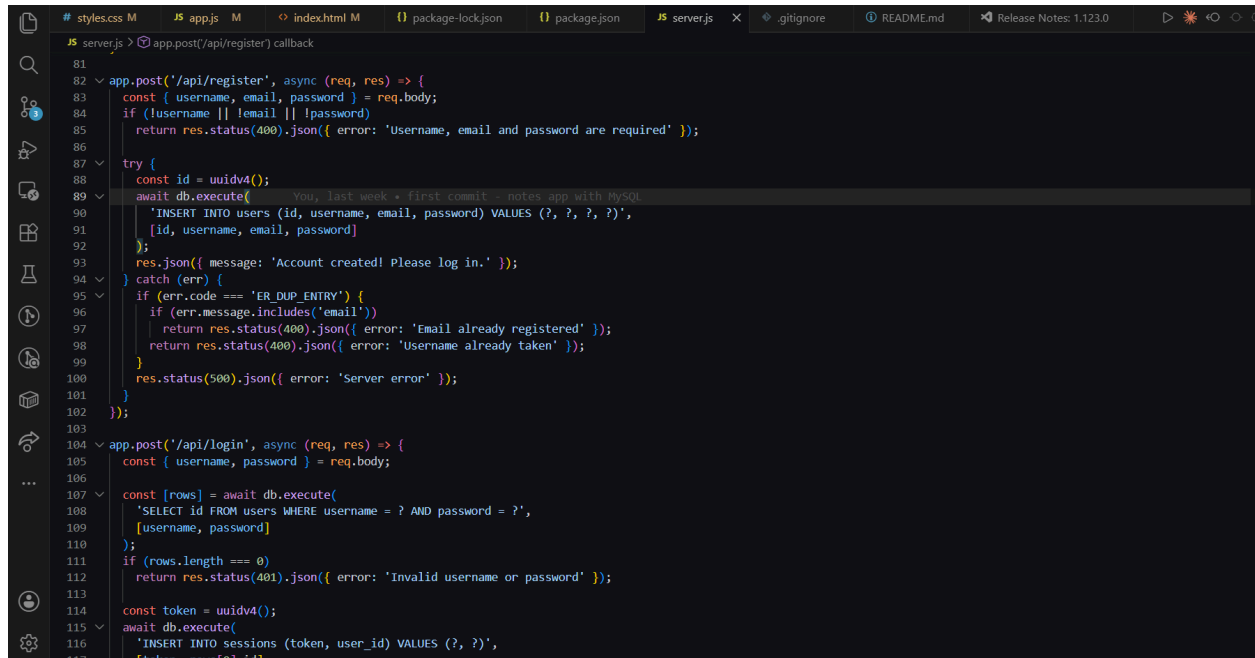
Task 4 - Backend Folder Organization

Fig 1: Project Structure



Task 5 - Introduction to Modern Backend Alternatives

Fig : PHP Modern Alternative ([Node.js](https://nodejs.org/))

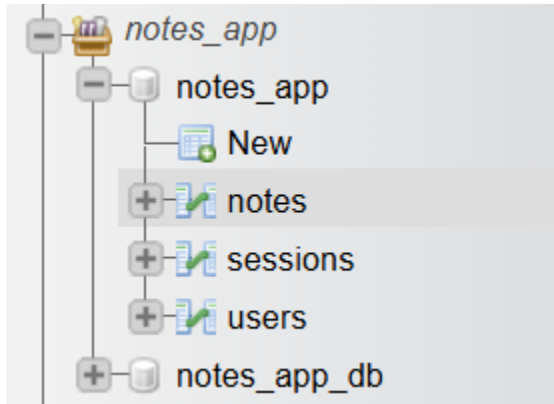


```
server.js > app.post('/api/register') callback
81
82 app.post('/api/register', async (req, res) => {
83   const { username, email, password } = req.body;
84   if (!username || !email || !password)
85     return res.status(400).json({ error: 'Username, email and password are required' });
86
87   try {
88     const id = uuidv4();
89     await db.execute(
90       'INSERT INTO users (id, username, email, password) VALUES (?, ?, ?, ?)',
91       [id, username, email, password]
92     );
93     res.json({ message: 'Account created! Please log in.' });
94   } catch (err) {
95     if (err.code === 'ER_DUP_ENTRY') {
96       if (err.message.includes('email'))
97         return res.status(400).json({ error: 'Email already registered' });
98       return res.status(400).json({ error: 'Username already taken' });
99     }
100     res.status(500).json({ error: 'Server error' });
101   }
102 });
103
104 app.post('/api/login', async (req, res) => {
105   const { username, password } = req.body;
106
107   const [rows] = await db.execute(
108     'SELECT id FROM users WHERE username = ? AND password = ?',
109     [username, password]
110   );
111   if (rows.length === 0)
112     return res.status(401).json({ error: 'Invalid username or password' });
113
114   const token = uuidv4();
115   await db.execute(
116     'INSERT INTO sessions (token, user_id) VALUES (?, ?)',
117     [token, rows[0].id]
```

Week 5 - Advanced Web Design and Development

Task 1: Database Creation

Fig 1: notes-app database



Task 2: Creating Tables

Fig 1: notes-app tables

Table	Action	Rows	Type	Collation	Size	Overhead
☐ notes	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ sessions	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	48.0 KiB	-
3 tables	Sum	8	InnoDB	utf8mb4_general_ci	112.0 KiB	0 B

Task 3: CRUD Operations

Fig 1: Add Records (INSERT)

				id	user_id	title	content	created_at
				14f49b21-31f0-4204-aafd-8f7d2e02a11a	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Machine Learning	How Kenya has integrated Machine Learning	2026-06-03 12:37:30
				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14
		Check all	With selected:					

Fig 2: View Records (SELECT)

NoteKeeper

MY NOTES3

Machine Learning

How Kenya has integrated Machine Le...

Read Hacker News

Go to the latest article of The Hac...

5/28/2026

TryHackMe

Do labs on TryHackMe to better unde...

5/21/2026

New Note

CREATE NOTE

Title

Note title...

Content

Write your note here...

Save Note

Fig 3: Delete Records (DELETE)

				id	user_id	title	content	created_at
☐				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14

Fig 4: Edit Records (UPDATE)

				id	user_id	title	content	created_at
☐				9d568126-c31e-45a2-b458-8d103f3f9c43	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	TryHackMe TODAY	Do labs on TryHackMe to better understanding	2026-05-21 13:25:24
☐				9f938a0c-ff96-4a76-bdc0-674a8319eb31	635e5f65-f7fe-4e86-b56d-9a9086dfcf09	Read Hacker News	Go to the latest article of The Hacker News	2026-05-28 14:33:14
	☐	Check all	With selected:					

Task 4: Connecting PHP to Database

Fig 1: Database Connection

```
const DB_CONFIG = {
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: '',
  database: 'notes_app',
  multipleStatements: true
};

let db;

async function initDB() {
  const tempConn = await mysql.createConnection({
    host: DB_CONFIG.host,
    port: DB_CONFIG.port,
    user: DB_CONFIG.user,
    password: DB_CONFIG.password
  });

  await tempConn.execute(`CREATE DATABASE IF NOT EXISTS \`${DB_CONFIG.database}\``);
  await tempConn.end();

  db = await mysql.createPool(DB_CONFIG);
}
```

Task 5: Fetching Database Records

Fig 1: Database Records

```
app.put('/api/notes/:id', requireAuth, async (req, res) => {
  const { title, content } = req.body;
  if (!title || !content)
    return res.status(400).json({ error: 'Title and content required' });

  await db.execute(
    'UPDATE notes SET title = ?, content = ? WHERE id = ? AND user_id = ?',
    [title, content, req.params.id, req.userId]
  );
  res.json({ message: 'Note updated!' });
});
```

Creativity: Note Counter

Fig 1: shows the dynamic note count badge in the sidebar header displaying the total number of notes.

